

396. Rotate Function

You are given an integer array `nums` of length `n`.

Assume `arr k` to be an array obtained by rotating `nums` by `k` positions clock-wise. We define the **rotation function** `F` on `nums` as follow:

$$\bullet \text{ } F(k) = 0 * \text{arr } k[0] + 1 * \text{arr } k[1] + \dots + (n - 1) * \text{arr } k[n - 1].$$

Return the maximum value of `F(0)`, `F(1)`, ..., `F(n-1)`.

The test cases are generated so that the answer fits in a **32-bit** integer.

-

Example 1:

Input: `nums = [4,3,2,6]`

Output: `26`

Explanation:

$$F(0) = (0 * 4) + (1 * 3) + (2 * 2) + (3 * 6) = 0 + 3 + 4 + 18 = 25$$

$$F(1) = (0 * 6) + (1 * 4) + (2 * 3) + (3 * 2) = 0 + 4 + 6 + 6 = 16$$

$$F(2) = (0 * 2) + (1 * 6) + (2 * 4) + (3 * 3) = 0 + 6 + 8 + 9 = 23$$

$$F(3) = (0 * 3) + (1 * 2) + (2 * 6) + (3 * 4) = 0 + 2 + 12 + 12 = 26$$

So the maximum value of `F(0)`, `F(1)`, `F(2)`, `F(3)` is `F(3) = 26`.

Example 2:

Input: `nums = [100]`

Output: `0`

-

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 10^5$
- $-100 \leq \text{nums}[i] \leq 100$

DP Recurrence Detailed Explantion | Beats 100%

5625

Array

Math

Dynamic Programming

Python

4+

```
class Solution {  
  
    public int maxRotateFunction(int[] A) {  
  
        int sum = 0, F = 0;  
  
        int n = A.length;  
  
  
        for (int i = 0; i < n; i++) {  
  
            sum += A[i];  
  
            F += i * A[i];  
  
        }  
  
        int max = F;  
  
  
        for (int i = 1; i < n; i++) {  
  
            F += sum - n * A[n - i];  
  
            max = Math.max(max, F);  
  
        }  
  
        return max;  
  
    }  
}
```

Intuition

We need a fast way to calculate the next rotation value, $F(i)$, using the previous value, $F(i-1)$.

This avoids recalculating the entire sum from scratch at every step.

Let's consider the array A :

$$A = [a, b, c, \dots, y, z]$$

with a length of n , where z is the last element of the array:

$$z = A[n-1]$$

Approach

Initial State:

0	1	2	...	$n-3$	$n-2$	$n-1$
a	b	c	...	x	y	z

Let the initial state be $F(0)$, where each element is multiplied by its corresponding index:

$$F(0) = 0a + 1b + 2c + \dots + (n-2)y + (n-1)z$$

By performing clockwise rotation by 1, the next state becomes:

0	1	2	3	...	$n-2$	$n-1$
z	a	b	c	...	x	y

Thus, $F(1)$:

$$F(1) = 1a + 2b + 3c + \dots + (n-1)y + 0z$$

To find out what changes between steps, let's determine the difference between $F(1)$ and $F(0)$:

$$\begin{aligned} F(1) - F(0) &= 1a + 2b + 3c + \dots + (n-1)y + 0z - [0a + 1b + 2c + \dots + (n-2)y + (n-1)z] \\ &= 1a + 1b + 1c + \dots + 1y - (n-1)z \end{aligned}$$

Hence, the difference between two states is:

$$\text{diff} = a + b + c + \dots + y - (n-1)z$$

Now, if we add $+z$, we can convert the inner expression into the sum of the array $Asum$:

$$diff = Asum + b + c + \dots + y + z - (n-1)z$$

Then, add $-z$ back to restore the equation, and simplify:

$$diff = Asum + (-z) - (n-1)z = Asum - z - zn + z = Asum - nz = Asum - n \times A[n-1] \text{ Substitute: } z = A[n-1]$$

Therefore:

$$diff = Asum - A[n-1] \cdot n$$

Recurrence Relation

Determining the next state $F(1)$ using current state $F(0)$:

Since $F(1) - F(0) = diff$, we add $F(0)$ to both sides:

$$F(1) - F(0) = diff \implies F(1) = F(0) + diff = F(0) - Asum + A[n-1] \cdot n$$

Generalization

We can generalize the formula to any $F(i)$:

If $F(i)$ is the current state, then $F(i+1)$ is the next state, and we can apply the same rule:

For example, in $F(1)$, the current array state is:

<i>original</i>	$n-1$	0	1	2	...	$n-3$	$n-2$
<i>rotated</i>	0	1	2	3	...	$n-2$	$n-1$
	z	a	b	c	...	x	y

where the last element is:

$$y = A[n-2]$$

Then the next last element for the following states are:

<i>original</i>	$n-2$	$n-1$	0	1	2	...	$n-3$
<i>rotated</i>	0	1	2	3	4	...	$n-1$
	y	z	a	b	c	...	x

<i>original</i>	$n-3$	$n-2$	$n-1$	0	1	...	$n-4$
<i>rotated</i>	0	1	2	3	4	...	$n-1$
	x	y	z	a	b	...	w

<i>original</i>	$n-4$	$n-3$	$n-2$	$n-1$	0	...	$n-5$
<i>rotated</i>	0	1	2	3	4	...	$n-1$
	w	x	y	z	a	...	v

Hence:

<i>original</i>	$n-5$	$n-4$	$n-3$	$n-2$	$n-1$	...	$n-i$
<i>rotated</i>	0	1	2	3	4	...	$n-1$
	v	w	x	y	z	...	next

$$\text{next} = A[n-i]$$

Conclusion

For any step i , the pattern is:

$$F(i) = F(i-1) + \text{Asum} - A[n-i] \cdot n$$

We use this simple formula to calculate all rotation values using only the total sum and the single element that moved.

Time Complexity: $O(n)$

Space Complexity: $O(1)$